OSCORE-capable Proxies

draft-ietf-core-oscore-capable-proxies-07

Marco Tiloca, RISE
Rikard Höglund, RISE

IETF 126 Meeting – Vienna – July 23rd, 2026

Scope: update RFC8613 and RFC8768

- **Define the use of OSCORE in a communication leg including a proxy**
  - Between origin client/server and a proxy; or between two proxies in a chain
  - Not only an origin client/server, but also an intermediary can be an “OSCORE endpoint”
- **Define rules to escalate the protection of CoAP options**
  - If possible, encrypt and integrity-protect an option originally defined as Class U or I for OSCORE
- **Explicitly admit nested OSCORE protection**
  - For example, first protect end-to-end over C ↔ S, then further protect the result over C ↔ P
  - Typically, at most 2 OSCORE “layers” for the same message
    - 1 end-to-end + 1 between two adjacent hops
  - Possible to seamlessly apply 2 or more OSCORE layers to the same message
- **Explicitly define the Hop-Limit Option (RFC 8768) to be Class U for OSCORE**

Focus on OSCORE, but the same applies “as is” to Group OSCORE

Updates since version -06

- **Mostly about two points left from Christian's review of version -04 – Thanks!**
  - https://mailarchive.ietf.org/arch/msg/core/IZsAWLHd8xHu78syUh78OfwWEHM/
  - The other points were addressed in version -06 and presented at IETF 125

- **Detailed reply on the CoRE mailing list**
  - https://mailarchive.ietf.org/arch/msg/core/kT9YyIDncR6sqRoHcF82-AYD0Gc/

- **The two points are:**
  - Section 2.4 "Processing of an Incoming Request"
    - Improve the presentation of the algorithm, e.g., without repeating word-by-word the behavior that is common to both forward-proxy and reverse-proxy
  - Any guideline for a proxy adjacent to the origin server, about when trying to establish/use OSCORE with the origin server?

Processing of an incoming request

- **Section 2.4 – Major revision of the presentation of the algorithm**
  - No change in the processing rationale, actions, and outcome
  - Countercheck: no impact in the corresponding state diagram in Appendix D

- **Gist of the revised algorithm presentation**
  1. If the request processing is potentially about proxying → Step 2. Otherwise → Step 5.
  2. If confirmed to be about forward-proxying and doable → Step 4. Otherwise → Step 3.
  3. If confirmed to be about reverse-proxying and doable → Step 4. Otherwise → Step 5.
  4. Do error checking/handling that is common to forward- and reverse-proxies.
     - If there are issues, fail. Otherwise,
     - Forward the request; specific differences apply to forward- or reverse-proxy.
  5. If no OSCORE Option, give to the application (if any) and stop. Otherwise → Step 6.
  6. Decrypt. If there are issues → fail. Otherwise → Step 1.

Guidance to the proxy

- **New Section 5.1 “Guidelines for Proxies”**
  - Q0: When should the proxy P establish/use OSCORE with the origin server S?

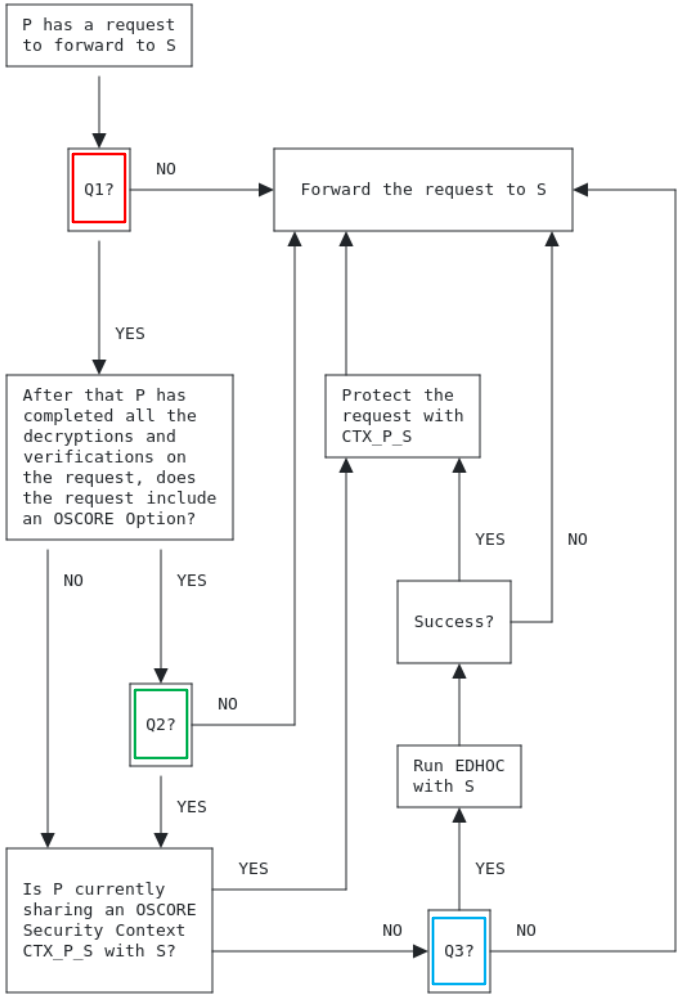
- **The proxy can answer three questions:**
  - Q1: Does S wish to use OSCORE between itself and an adjacent proxy?
    - P could rely on an SvcParamKey within a DNS SVCB Resource Record about S (*)
  - Q2: Does S support the use of multiple, nested OSCORE layers?
    - P could rely on an SvcParamKey within a DNS SVCB Resource Record about S (*)
  - Q3: Does S support the key exchange protocol EDHOC (RFC 9528)?
    - P could use the means defined in *draft-ietf-lake-app-profiles*

- **The answer to Q0 comes from the answers to (Q1, Q2, Q3), see next slide**

(*) Should we actually define and register the two SvcParamKeys above?

Guidance to the proxy

- **Q1**: Does S wish to use OSCORE between itself and an adjacent proxy?
- **Q2**: Does S support the use of multiple, nested OSCORE layers?
- **Q3**: Does S support the key exchange protocol EDHOC (RFC 9528)?



New example of message exchange

- **New Appendix B.6, with two forward-proxies P1 and P2**
  - Client C --- Proxy P1 --- Proxy P2 --- Server S
  - Assuming all the OSCORE Security Contexts already established
  - OSCORE between:
    - C and S
    - C and P1
    - P1 and P2
    - P2 and S

- **See the AASVG figure for details**

Implementation

- **Eclipse Californium (Java) – Lucas Åhl**
  - https://github.com/Toflowz/californium/tree/for-Edhoc
  - Tested with the Uri-Proxy Option or with the Proxy-Scheme Option plus the Uri-* Options
  - Tested with OSCORE over Client-Proxy and/or Client-Server and/or Proxy-Server
  - Tested also using Observe (RFC 7641) and key exchange with EDHOC (RFC 9528)

- **Contiki-NG – Manassanan Vongprai**
  - https://github.com/depuf/contiki-ng/tree/develop
  - Tested on Zolertia FireFly nodes
  - Tested with the Uri-Proxy Option
  - Tested with OSCORE over Client-Proxy and Client-Server

- **The two implementations successfully interoperated**

Next steps

- **Handling replay and local ordering of multiple responses to the same request**
  - Same rationale and approach as in Group OSCORE, see *draft-ietf-core-oscore-groupcomm*
- **Add the “Operational Considerations” section**
- **Comments and reviews are welcome!**

Thank you!

Comments/questions?

https://github.com/core-wg/oscore-capable-proxies